Vacuolar phenotype analyzer

Automated quantification of vacuolar phenotypes (A/B/C) in yeast from confocal FM4-64 images.

Contents

- `vacuolar_pipeline.py` : End-to-end pipeline to extract crops, train a classifier, and apply it to LIF or OME-TIFF/TIFF images.
- `apply_filter.py` : Runs inference on LIF files with confidence filtering and writes a per-image CSV (raw + filtered counts).
- `Model.keras` : Trained CNN classifier exported as a Keras model.
- `Model_class_map.csv` : Label mapping for the model.
- `config.ps1` : Template PowerShell config to rerun the analysis with consistent paths and parameters.
- `inputs_lif.txt` : List of LIF inputs (one path per line).
- `inputs_ome_dir.txt` : Folder path for OME-TIFF/TIFF inputs.
- `inputs_params.txt` : Optional parameters (min_conf/min_margin, etc.) for config.ps1.

Model generation (training)

A total of 2,321 single-yeast-cell image crops were manually labeled as A/B/C from confocal FM4-64 z-stacks (OME-TIFF/LIF). FM4-64 is a lipophilic dye that labels vacuolar membranes (excitation/emission: 515–640 nm). Training and testing images were acquired on a Leica SP8 confocal microscope (Leica Microsystems, Germany), using a 63X objective with 5X magnification. Samples covered multiple conditions, selected to ensure representation of all three phenotypes within the labeled pool. Z-stacks were converted to 2D images by maximum-intensity projection along the z-axis. Candidate cells were detected using a blob-based detector with fixed parameters (crop size: 96 × 96 px; threshold: 0.06; sigma_min: 3; sigma_max: 12). The final model was trained on 900 crops selected from the labeled pool using a cap_max sampling strategy (seed = 42), which yielded the best validation performance. Crops were classified into vacuolar phenotypes A/B/C using a convolutional neural network (CNN), and the trained model was exported as a Keras .keras file.

Vacuolar phenotype definitions (A/B/C)

Summary used for labeling:

- Phenotype A: lowest vacuolar fragmentation; typical in normal conditions; cells with up to 2 vacuoles.
- Phenotype B: 3 or more clearly identifiable vacuoles.
- Phenotype C: highly fragmented vacuoles that are difficult to quantify by number.

Seeley, E. S., Kato, M., Margolis, N., Wickner, W., & Eitzen, G. (2002). Genomic analysis of homotypic vacuole fusion. Molecular Biology of the Cell, 13(3), 782–794. https://doi.org/10.1091/mbc.01-10-0512

Training environment: TensorFlow 2.16.1 / Keras 3.0.5. Original inference environment during development: TensorFlow 2.15.0 / Keras 2.15.0 (legacy model format).

Model file compatibility

The provided `Model.keras` was saved with Keras 3.x. It cannot be loaded by Keras 2.x (e.g., TensorFlow/Keras 2.15), and will raise a version mismatch error. For inference with `Model.keras`, use a Keras 3 environment (TensorFlow >= 2.16).

Recommended Keras 3 setup (tested with Keras 3.13.0 / TensorFlow 2.16.1):

```
python -m venv .venv-keras3
. .\.venv-keras3\Scripts\Activate.ps1
pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
         readlif pillow
```

If you must use Keras 2.15 for inference, re-save the model from a Keras 3 environment into a legacy format (e.g., `Model.h5`) and point the pipeline to that file instead.

Quick start (Windows PowerShell, from zero)

Use these steps if this is your first time running the model.

> **Note**: Directory paths in the commands below (such as `D:\Vacuolar-phenotype-analizer` or `C:\vacuolar\...`) are reference examples. You must replace them with the actual paths to your working directory where you downloaded the repository and where your files are located to run them successfully.

1. Install Python (if needed)

```
python --version
```

If `python` is not recognized:

```
winget install -e --id Python.Python.3.11
```

Then close and reopen PowerShell.

If venv activation is blocked, run once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

2. Open PowerShell in this repository

```
Set-Location "D:\Vacuolar-phenotype-analizer"
```

3. Create and activate a virtual environment (recommended for `Model.keras`)

```
python -m venv .venv-keras3
. .\.venv-keras3\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
         readlif pillow
```

4. Confirm packages were installed in the active environment

```
python -c "import tensorflow, keras; print('tensorflow', tensorflow.__version__, '|
         keras', keras.__version__)"
```

5. Set your input files

- Edit `inputs_lif.txt` with one `.lif` path per line (specify the exact directory path where your `.lif` files are located).
- Optional: edit `inputs_params.txt` to change `min_conf`, `min_margin`, or set `disable_filter=true`.

6. Run the analysis

- If the environment is already active:

```
.\config.ps1
```

- If the environment is not active, run everything in one line:

```
Set-Location "D:\Vacuolar-phenotype-analizer"; . .\.venv-
        keras3\Scripts\Activate.ps1; .\config.ps1
```

Outputs are written to `outputs\`.

Environment and dependencies

Commands are intended to be run in Windows PowerShell (5.1 or 7) from a Python virtual environment.

Dependencies:

- Python 3.x
- `tensorflow` (2.16.1 recommended for `Model.keras`; 2.15.0 only for legacy models such as `.h5`)
- `numpy`, `pandas`
- `tifffile`, `scikit-image`
- `readlif`, `pillow` (only for `.lif`)

Example setup for inference with the provided `Model.keras`:

```
python -m venv .venv-keras3
. .\.venv-keras3\Scripts\Activate.ps1
pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
         readlif pillow
```

Legacy setup (only if you converted the model to a Keras 2-compatible format, such as `.h5`):

```
python -m venv .venv
. .\.venv\Scripts\Activate.ps1
pip install tensorflow==2.15.0 numpy pandas tifffile scikit-image readlif pillow
```

macOS (PowerShell or Bash)

You can run the same Python commands in macOS. There are two options:

1. With PowerShell installed:

```
python3 -m venv .venv
source .venv/bin/activate
pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
         readlif pillow
pwsh ./config.ps1
```

2. Without PowerShell: run the Python commands directly (see sections below).

Windows vs macOS differences:

- Windows venv activation: `. .\.venv\Scripts\Activate.ps1`
- macOS venv activation: `source .venv/bin/activate`
- Path separators: Windows `C:\path\to\file` vs macOS `/path/to/file`

Apply the model

LIF with confidence filtering (apply_filter.py)

This route calls the `extract` subcommand from a pipeline script to generate crops, then classifies them and filters uncertain predictions.

PowerShell example:

```
$files = @(
  "C:\vacuolar\data\lif\sample_01.lif"
)

$model = "C:\vacuolar\models\Model.keras"
$outDir = "C:\vacuolar\outputs"
$pipeline = "C:\vacuolar\vacuolar_pipeline.py"

if (-not (Test-Path $outDir)) { New-Item -ItemType Directory -Path $outDir | Out-
         Null }

# Ensure the class map sits next to the model as "<model>_class_map.csv"
$classMapSrc = "C:\vacuolar\models\Model_class_map.csv"
if (-not (Test-Path $classMapSrc)) { throw "Missing class map: $classMapSrc" }

foreach ($f in $files) {
  $name = [System.IO.Path]::GetFileNameWithoutExtension($f)
  $outCsv = Join-Path $outDir ("conteo_por_imagen_val_{0}_DUAL.csv" -f $name)

  python "C:\vacuolar\apply_filter.py" `
    --lif_file "$f" `
    --model "$model" `
    --out_csv "$outCsv" `
    --pipeline_py "$pipeline" `
    --crop_size 96 --threshold 0.06 --sigma_min 3 --sigma_max 12 `
    --batch_size 128 `
    --min_conf 0.60 --min_margin 0.10
}
```

To disable filtering and keep raw counts only, add `--disable_filter`. When this flag is set, `min_conf` and `min_margin` are ignored. When using `config.ps1`, set `disable_filter=true` in `inputs_params.txt` instead.

Dependencies (Python) for this step:

- `tensorflow>=2.16`, `keras>=3`, `numpy`, `pandas`, `tifffile`, `scikit-image`, `readlif`, `pillow` (for `Model.keras`)

OME-TIFF/TIFF folder (or a single OME-TIFF)

`vacuolar_pipeline.py apply` processes all `.tif/.tiff/.ome.tif/.ome.tiff` files in a directory. For a single image, put the file in its own folder and point `--images_dir` to that folder.

```
python "C:\vacuolar\vacuolar_pipeline.py" apply `
  --images_dir "C:\vacuolar\data\ome_tif" `
  --model "C:\vacuolar\models\Model.keras" `
  --out_csv "C:\vacuolar\outputs\conteo_por_imagen.csv" `
  --crop_size 96 --threshold 0.06 --sigma_min 3 --sigma_max 12 `
  --batch_size 128
```

This command already outputs raw counts; there is no confidence filtering or `--disable_filter` option here.

Dependencies (Python) for this step:

- `tensorflow>=2.16`, `keras>=3`, `numpy`, `pandas`, `tifffile`, `scikit-image` (for `Model.keras`)

Notes on inputs

- Z-stacks are converted to 2D by maximum-intensity projection.
- Images are expected to have 2 channels; 1-channel inputs are duplicated. Extra channels are truncated to the first two.

Template config.ps1

Edit `inputs_lif.txt` (and optionally `inputs_ome_dir.txt`) to set your input paths, then run `config.ps1` to reproduce the LIF inference loop. To change `min_conf` and `min_margin` without editing the script, edit `inputs_params.txt`. Relative paths in those files are resolved from the repository root.

Note: the script name is `config.ps1`.

```
.\config.ps1
```

If your virtual environment is not active, use:

```
Set-Location "D:\Vacuolar-phenotype-analizer"; . .\.venv-
         keras3\Scripts\Activate.ps1; .\config.ps1
```

Example dataset layout

```
data/
  lif/
    sample_01.lif
  ome_tif/
    image_01.ome.tif
inputs_lif.txt
inputs_ome_dir.txt
inputs_params.txt
models/
  Model.keras
  Model_class_map.csv
outputs/
  conteo_por_imagen.csv
```

Parameters: min_conf and min_margin

- `min_conf`: Minimum softmax probability for the top-1 class. Crops below this value are excluded from the filtered counts.
- `min_margin`: Minimum gap between the top-1 and top-2 probabilities. This removes ambiguous crops where the model is not clearly confident.
- Use `--disable_filter` to keep all detected crops (filtered columns equal raw columns).

Common errors and quick fixes

- `python` is not recognized: Install Python and reopen PowerShell.

  ```
  winget install -e --id Python.Python.3.11
  ```

- Activation blocked (`running scripts is disabled on this system`): Allow local scripts for your user, then activate again.

  ```
  Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
  . .\.venv-keras3\Scripts\Activate.ps1
  ```

- `ModuleNotFoundError` or missing packages: You are likely outside the virtual environment, or packages were installed globally instead of inside the environment.

  ```
  . .\.venv-keras3\Scripts\Activate.ps1
  python -m pip install --upgrade pip
  pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
           readlif pillow
  ```

- Keras/TensorFlow version mismatch when loading `Model.keras`: The provided model requires Keras 3. Use TensorFlow >= 2.16 and Keras >= 3.

  ```
  python -c "import tensorflow, keras; print(tensorflow.__version__,
           keras.__version__)"
  ```

  If you must run TensorFlow 2.15/Keras 2.15, convert and use a legacy model format (for example, `Model.h5`).

- `Missing model file` or class map errors: Ensure these files exist and keep standard names:

  - `models\Model.keras`
  - `models\Model_class_map.csv`

- `Skipping missing file` warnings from `config.ps1`: One or more paths in `inputs_lif.txt` do not exist. Fix the path(s) and rerun.

- Runtime memory issues (slow or crashes during inference): Reduce `batch_size` in `inputs_params.txt` (for example, `batch_size=32`), then run `config.ps1` again.
- Outputs are not where expected: By default, `config.ps1` writes CSVs to `outputs\` inside the repository root.

Quick diagnostics (with fixes)

Run these checks when setup or execution is not working as expected.

1. Check Python is available

```
python --version
```

- If this fails: install Python and reopen PowerShell.

```
winget install -e --id Python.Python.3.11
```

2. Check which Python executable is being used

```
where python
```

- If the first path is not inside this repo virtual environment (for example, `.venv-keras3\Scripts\python.exe`), activate the environment:

```
Set-Location "D:\Vacuolar-phenotype-analizer"
. .\.venv-keras3\Scripts\Activate.ps1
```

3. Check pip belongs to the same environment

```
pip -V
```

- If pip points to global/site packages, reinstall inside the active environment:

```
python -m pip install --upgrade pip
pip install tensorflow==2.16.1 keras==3.13.0 numpy pandas tifffile scikit-image
         readlif pillow
```

4. Check TensorFlow/Keras import and versions

```
python -c "import tensorflow, keras; print('tensorflow', tensorflow.__version__, '|
         keras', keras.__version__)"
```

- If import fails: reinstall dependencies in the active environment.
- If versions are below TensorFlow 2.16 / Keras 3: upgrade packages or recreate `.venv-keras3`.

5. Check required files before running

```
Test-Path .\config.ps1
Test-Path .\models\Model.keras
Test-Path .\models\Model_class_map.csv
```

- If any result is `False`, fix file paths or restore missing files.

6. Check inputs list and run

```
Get-Content .\inputs_lif.txt
.\config.ps1
```

- If warnings show missing files, correct those paths in `inputs_lif.txt`.

Citation

If you use this software, please cite:

Szenfeld, B. (2025). *Vacuolar Phenotype Analyzer*. Zenodo.
DOI: https://doi.org/10.5281/zenodo.18124375

Contact

For any questions or inquiries, you can reach out via email: bioszenf@gmail.com